

```
"""
```

```
Shadow Node Theory v2.0 – Operacionalizacion del Indice de Soberania Atomica (ASI  
Fractal Core Research | Tlaxcala, Mexico | 2026
```

```
El ASI mide la capacidad del Nodo Atomico de resolver la entropia de su entorno.  
Un nodo con ASI > 1 resuelve problemas nuevos con la misma eficacia que los conoc  
adopta herramientas espontaneamente y mantiene desempeno consistente bajo presion
```

```
Formula operacional:
```

$$\text{ASI} = (\text{delta_H} * \alpha) / F$$

```
Donde:
```

```
delta_H : diversidad de tipos de eventos resueltos exitosamente  
          (Shannon entropy de la distribucion de eventos por usuario)  
alpha    : autonomia de decision = proporcion de herramientas adoptadas  
          espontaneamente vs reactivamente (sin trigger de onboarding)  
F         : entropia interna = coeficiente de variacion del desempeno  
          entre sesiones (std / mean de eventos por dia activo)
```

```
Fuentes:
```

- zerve_hackathon_dataset.csv (409,287 eventos, 4,774 usuarios)
- user_metrics_fractal (precomputado en pipeline Fractal V3)

```
Uso:
```

```
python snt_así_operacionalizacion.py --input zerve_hackathon_dataset.csv
```

```
Salida:
```

- snt_así_scores.csv : ASI por usuario con componentes
- snt_así_distribucion.png : distribucion ASI por cohorte
- snt_así_validacion.png : ASI vs CSI_V3 + correlaciones

```
"""
```

```
import numpy as np  
import pandas as pd  
from scipy.stats import entropy as shannon_entropy, pearsonr, spearmanr  
from scipy.optimize import curve_fit  
import matplotlib.pyplot as plt  
import matplotlib.gridspec as gridspec  
import warnings  
import sys  
import os  
warnings.filterwarnings('ignore')
```

```
# =====
```

```

# CONFIGURACION
# =====

INPUT_FILE = "zerve_hackathon_dataset.csv"
OUTPUT_CSV = "/home/claude/snt_asl_scores.csv"
OUTPUT_PNG_DIST = "/home/claude/snt_asl_distribucion.png"
OUTPUT_PNG_VAL = "/home/claude/snt_asl_validacion.png"

plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'font.size': 9,
    'figure.dpi': 150,
    'axes.spines.top': False,
    'axes.spines.right': False,
})

BG = '#1D1D20'
TXT = '#fbfbff'
SEC = '#909094'

# Clasificacion de eventos por tipo de autonomia
# Eventos de ONBOARDING (reactivos - el sistema los triggerea)
EVENTOS_REACTIVOS = {
    'canvas_onboarding_tour_started',
    'canvas_onboarding_tour_add_block_step',
    'canvas_onboarding_tour_ai_assistant_step',
    'canvas_onboarding_tour_code_and_variables_step',
    'canvas_onboarding_tour_compute_step',
    'canvas_onboarding_tour_running_blocks_step',
    'canvas_onboarding_tour_you_are_ready_step',
    'canvas_onboarding_tour_finished',
    'skip_onboarding_form',
    'submit_onboarding_form',
    'new_user_created',
    'sign_up',
    'sign_in',
    'ai_credit_banner_shown',
    'ai_credit_banner_clicked',
    'email_mobile_link_clicked',
    'link_clicked',
    'quickstart_add_dataset',
    'quickstart_explore_playground',
    'quickstart_upload_file',
}

# Eventos AUTONOMOS (el usuario los inicia por cuenta propia)
EVENTOS_AUTONOMOS = {

```

```

'run_block',
'run_all_blocks',
'run_from_block',
'run_upto_block',
'block_create',
'block_delete',
'block_copy',
'block_paste',
'block_rename',
'edge_create',
'edge_delete',
'canvas_create',
'canvas_clone',
'canvas_publish_to_community',
'agent_open',
'agent_message',
'agent_new_chat',
'agent_tool_call_create_block_tool',
'agent_tool_call_run_block_tool',
'agent_tool_call_get_block_tool',
'agent_accept_suggestion',
'agent_refactor_block',
'agent_web_search',
'agent_voice_to_text',
'agent_start_from_prompt',
'api_deploy',
'app_create',
'app_publish',
'scheduled_job_start',
'source_control_commit',
'layer_create',
'layer_duplicate',
'hosted_apps_deploy',
'requirements_build',
'folder_create',
'files_upload',
'notebook_import',
}

```

```

# =====
# CARGA DE DATOS
# =====

```

```

print("=" * 70)
print("SNT v2.0 – OPERACIONALIZACION DEL ASI (Indice de Soberania Atomica)")
print("Fractal Core Research | 2026")
print("=" * 70)

```

```

if not os.path.exists(INPUT_FILE):
    print(f"\nARCHIVO NO ENCONTRADO: {INPUT_FILE}")
    print("Generando datos simulados para demostracion del modelo...")
    MODO_SIMULACION = True
else:
    print(f"\nCargando {INPUT_FILE}...")
    df_raw = pd.read_csv(INPUT_FILE)
    df_raw['timestamp'] = pd.to_datetime(df_raw['timestamp'], format='mixed', utc=True)
    print(f"Filas: {len(df_raw):,} | Usuarios: {df_raw['person_id'].nunique():,}")
    MODO_SIMULACION = False

# =====
# COMPUTACION DE COMPONENTES DEL ASI
# =====

if not MODO_SIMULACION:

    print("\nComputando componentes del ASI...")
    df_sorted = df_raw.sort_values(['person_id', 'timestamp'])

    # -----
    # COMPONENTE 1: delta_H – Reduccion de incertidumbre
    # Shannon entropy de la distribucion de eventos por usuario
    # Alta entropia = el usuario usa muchos tipos distintos de herramientas
    # con distribucion equilibrada = resuelve entropia diversa
    # -----
    print("  Computando delta_H (Shannon entropy de eventos)...")

    def calcular_entropia_usuario(group):
        counts = group['event'].value_counts().values.astype(float)
        probs = counts / counts.sum()
        return float(shannon_entropy(probs, base=2))

    entropia_por_usuario = df_sorted.groupby('person_id').apply(
        calcular_entropia_usuario
    ).rename('delta_H_raw')

    # Normalizar delta_H a [0, 1]
    h_max = entropia_por_usuario.max()
    h_min = entropia_por_usuario.min()
    delta_H = ((entropia_por_usuario - h_min) / (h_max - h_min + 1e-9)).rename('delta_H')

    # -----
    # COMPONENTE 2: alpha – Autonomia de decision
    # Proporción de eventos autonomos vs reactivos
    # alpha = eventos_autonomos / (eventos_autonomos + eventos_reactivos)

```

```

# Si alpha > 0.5 el usuario opera principalmente por iniciativa propia
# -----
print("  Computando alpha (autonomia de decision)...")

def calcular_autonomia(group):
    eventos = group['event'].values
    n_autonomos = sum(1 for e in eventos if e in EVENTOS_AUTONOMOS)
    n_reactivos = sum(1 for e in eventos if e in EVENTOS_REACTIVOS)
    total_clasificados = n_autonomos + n_reactivos
    if total_clasificados == 0:
        return 0.0
    return n_autonomos / total_clasificados

alpha = df_sorted.groupby('person_id').apply(calcular_autonomia).rename('alph

# -----
# COMPONENTE 3: F – Energia libre interna (caos)
# Coeficiente de variacion del desempeno entre dias activos
# F = std(eventos_por_dia) / (mean(eventos_por_dia) + 1)
# F alto = desempeno inconsistente = alto caos interno
# F bajo = desempeno consistente = baja energia libre
# -----
print("  Computando F (coeficiente de variacion del desempeno)...")

def calcular_energia_libre(group):
    group = group.copy()
    group['fecha'] = group['timestamp'].dt.date
    eventos_por_dia = group.groupby('fecha').size().values.astype(float)
    if len(eventos_por_dia) < 2:
        return 0.5 # Un solo dia activo = incertidumbre moderada
    mean_ev = eventos_por_dia.mean()
    std_ev = eventos_por_dia.std()
    # Coeficiente de variacion normalizado
    cv = std_ev / (mean_ev + 1e-9)
    # Normalizar a [0, 1] con tanh para evitar outliers
    return float(np.tanh(cv / 2))

F_raw = df_sorted.groupby('person_id').apply(calcular_energia_libre).rename('

# Ajustar F: minimo 0.01 para evitar division por cero
F = F_raw.clip(lower=0.01).rename('F')

# -----
# COMPUTAR ASI
# ASI = (delta_H * alpha) / F
# ASI > 1: soberania cognitiva operativa
# ASI < 1: satelizacion cognitiva

```

```

# -----
print("  Computando ASI...")

asi_df = pd.concat([delta_H, alpha, F], axis=1).fillna(0)
asi_df['delta_H'] = asi_df['delta_H'].clip(lower=0)
asi_df['alpha']   = asi_df['alpha'].clip(lower=0, upper=1)
asi_df['F']       = asi_df['F'].clip(lower=0.01)

asi_df['ASI_raw'] = (asi_df['delta_H'] * asi_df['alpha']) / asi_df['F']

# Normalizar ASI: escalar para que la mediana de usuarios Elite = 1.0
# Esto da significado empirico al umbral ASI > 1
asi_df['ASI'] = asi_df['ASI_raw'].round(6)

# -----
# METRICAS BASE PARA VALIDACION
# -----
print("  Computando metricas base para validacion...")

metricas_base = df_sorted.groupby('person_id').agg(
    total_eventos    = ('event', 'count'),
    herramientas     = ('prop_tool_name', 'nunique'),
    creditos         = ('prop_credits_used', 'sum'),
    dias_activos     = ('timestamp', lambda x: x.dt.date.nunique()),
    vida_util_dias   = ('timestamp', lambda x: (x.max() - x.min()).days),
)

# VDR simplificado (rotaciones por minuto)
def calcular_vdr(group):
    tools = group.sort_values('timestamp')
    tools = tools[tools['prop_tool_name'].notna()].copy()
    tools['prev_tool'] = tools.groupby('person_id')['prop_tool_name'].shift(1)
    tools['rotacion'] = (tools['prop_tool_name'] != tools['prev_tool']) & too
    tools['prev_ts'] = tools.groupby('person_id')['timestamp'].shift(1)
    tools['time_delta'] = (tools['timestamp'] - tools['prev_ts']).dt.total_se
    rot = tools[tools['rotacion']]
    if len(rot) == 0:
        return 0.0
    return rot['rotacion'].sum() / (rot['time_delta'].mean() / 60 + 1)

# CSI_V3 simplificado para validacion
def scale(s):
    if s.max() == s.min():
        return s * 0
    return (s - s.min()) / (s.max() - s.min())

s_tools = scale(metricas_base['herramientas'])

```

```

s_ret    = scale(metricas_base['vida_util_dias'])
vdr_proxy = scale(metricas_base['dias_activos']) # proxy simple

metricas_base['CSI_V3'] = (s_tools * 0.4 + s_ret * 0.3 + vdr_proxy * 0.3) * 1

# Cohort
p99_5 = metricas_base['CSI_V3'].quantile(0.995)
p93_1 = metricas_base['CSI_V3'].quantile(0.931)

def assign_cohort(score):
    if score >= p99_5: return 'Elite'
    elif score >= p93_1: return 'Intermediate'
    else: return 'Basic'

metricas_base['cohort'] = metricas_base['CSI_V3'].apply(assign_cohort)

# Combinar todo
resultado = metricas_base.join(asi_df, how='left').fillna({'ASI': 0, 'delta_H

else:
    # MODO SIMULACION: generar datos que ilustran el modelo
    print("\nGenerando datos simulados para ilustracion del ASI...")
    np.random.seed(42)
    n = 4774

    # Simular tres grupos
    n_elite = 24
    n_inter = 306
    n_basic = n - n_elite - n_inter

    # Elite: alta delta_H, alto alpha, bajo F
    dh_elite = np.random.beta(8, 2, n_elite)
    al_elite = np.random.beta(7, 3, n_elite)
    f_elite = np.random.beta(2, 8, n_elite) + 0.01

    # Intermediate: valores medios
    dh_inter = np.random.beta(5, 5, n_inter)
    al_inter = np.random.beta(4, 6, n_inter)
    f_inter = np.random.beta(4, 6, n_inter) + 0.01

    # Basic: baja delta_H, bajo alpha, alto F
    dh_basic = np.random.beta(2, 8, n_basic)
    al_basic = np.random.beta(2, 8, n_basic)
    f_basic = np.random.beta(7, 3, n_basic) + 0.01

    delta_H = np.concatenate([dh_elite, dh_inter, dh_basic])
    alpha = np.concatenate([al_elite, al_inter, al_basic])

```

```

F_val    = np.concatenate([f_elite, f_inter, f_basic])
cohort    = ['Elite'] * n_elite + ['Intermediate'] * n_inter + ['Basic'] * n_ba
csi_v3    = np.concatenate([
    np.random.uniform(55, 75, n_elite),
    np.random.uniform(19, 55, n_inter),
    np.random.uniform(0, 19, n_basic),
])

asi_raw = (delta_H * alpha) / F_val

resultado = pd.DataFrame({
    'delta_H': delta_H,
    'alpha': alpha,
    'F': F_val,
    'ASI_raw': asi_raw,
    'ASI': asi_raw,
    'CSI_V3': csi_v3,
    'cohort': cohort,
})

resultado.index = [f'user_{i:04d}' for i in range(n)]

# =====
# CALIBRACION DEL UMBRAL ASI = 1
# =====

# Calibrar: escalar ASI para que la mediana de Elite = 1.0
elite_mask = resultado['cohort'] == 'Elite'
if elite_mask.sum() > 0:
    mediana_elite = resultado.loc[elite_mask, 'ASI'].median()
    if mediana_elite > 0:
        resultado['ASI'] = resultado['ASI'] / mediana_elite
        print(f"\nCalibracion ASI: mediana Elite = {mediana_elite:.6f}")
        print(f"    ASI re-escalado: mediana Elite → 1.0")

resultado['ASI'] = resultado['ASI'].round(6)
resultado['soberania'] = resultado['ASI'] >= 1.0

# =====
# ESTADISTICAS
# =====

print("\n" + "=" * 70)
print("RESULTADOS DEL ASI")
print("=" * 70)

print(f"\nDistribucion general:")
print(f"    ASI media:    {resultado['ASI'].mean():.4f}")

```



```

print(f" ASI mediana: {resultado['ASI'].median():.4f}")
print(f" ASI max:      {resultado['ASI'].max():.4f}")
print(f" ASI min:      {resultado['ASI'].min():.4f}")
print(f" ASI > 1 (soberania cognitiva): {resultado['soberania'].sum():,} usuario

print(f"\nASI por cohorte:")
for cohort in ['Elite', 'Intermediate', 'Basic']:
    mask = resultado['cohorte'] == cohort
    n = mask.sum()
    if n > 0:
        asi_c = resultado.loc[mask, 'ASI']
        sob_c = resultado.loc[mask, 'soberania'].mean()
        print(f" {cohort:<14}: n={n:>5} | media={asi_c.mean():.4f} | mediana={as

print(f"\nCorrelacion ASI vs CSI_V3:")
r, p = pearsonr(resultado['ASI'].clip(upper=10), resultado['CSI_V3'])
rs, ps = spearmanr(resultado['ASI'].clip(upper=10), resultado['CSI_V3'])
print(f" Pearson r={r:.4f}, p={p:.6f}")
print(f" Spearman r={rs:.4f}, p={ps:.6f}")

print(f"\nComponentes del ASI por cohorte:")
print(f" {'Cohorte':<14} {'delta_H':>10} {'alpha':>10} {'F':>10} {'ASI':>10}")
print(f" {'-'*50}")
for cohort in ['Elite', 'Intermediate', 'Basic']:
    mask = resultado['cohorte'] == cohort
    if mask.sum() > 0:
        r_ = resultado[mask]
        print(f" {cohort:<14} {r_['delta_H'].mean():>10.4f} {r_['alpha'].mean():

# =====
# VALIDACION: ASI > 1 predice cohorte Elite/Intermediate?
# =====

print(f"\nValidacion predictiva:")
tp = ((resultado['soberania'] == True) & (resultado['cohorte'].isin(['Elite', 'Int
fp = ((resultado['soberania'] == True) & (resultado['cohorte'] == 'Basic')).sum()
tn = ((resultado['soberania'] == False) & (resultado['cohorte'] == 'Basic')).sum()
fn = ((resultado['soberania'] == False) & (resultado['cohorte'].isin(['Elite', 'In

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 el

print(f" True Positives (ASI>1 & Elite/Inter): {tp:>5}")
print(f" False Positives (ASI>1 & Basic): {fp:>5}")
print(f" True Negatives (ASI<1 & Basic): {tn:>5}")
print(f" False Negatives (ASI<1 & Elite/Inter): {fn:>5}")

```

```

print(f" Precision: {precision:.4f}")
print(f" Recall:    {recall:.4f}")
print(f" F1:        {f1:.4f}")

# =====
# VISUALIZACIONES
# =====

fig = plt.figure(figsize=(16, 12), facecolor=BG)
gs = gridspec.GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.35)

COLORES_COHORT = {
    'Elite': '#FFD700',
    'Intermediate': '#A1C9F4',
    'Basic': '#FF9F9B',
}

# Panel 1: Distribucion de ASI por cohorte
ax1 = fig.add_subplot(gs[0, :2])
ax1.set_facecolor(BG)

asi_clip = resultado['ASI'].clip(upper=5) # clipar outliers para visualizacion
for cohort, color in COLORES_COHORT.items():
    mask = resultado['cohort'] == cohort
    if mask.sum() > 0:
        ax1.hist(asi_clip[mask], bins=30, alpha=0.7, color=color,
                  label=f'{cohort} (n={mask.sum():,})', edgecolor='none')

ax1.axvline(x=1.0, color='white', linestyle='--', linewidth=2, alpha=0.8, label='
ax1.set_xlabel('ASI (Indice de Soberania Atomica)', color=SEC)
ax1.set_ylabel('Frecuencia', color=SEC)
ax1.set_title('Distribucion del ASI por Cohorte', color=TXT, fontweight='bold')
ax1.tick_params(colors=SEC)
ax1.legend(fontsize=8, facecolor='#2a2a2d', labelcolor=TXT)

# Panel 2: ASI vs CSI_V3 scatter
ax2 = fig.add_subplot(gs[0, 2])
ax2.set_facecolor(BG)

for cohort, color in COLORES_COHORT.items():
    mask = resultado['cohort'] == cohort
    if mask.sum() > 0:
        ax2.scatter(resultado.loc[mask, 'ASI'].clip(upper=5),
                     resultado.loc[mask, 'CSI_V3'],
                     c=color, alpha=0.5, s=10, label=cohort)

ax2.axvline(x=1.0, color='white', linestyle='--', linewidth=1, alpha=0.6)

```

```

ax2.set_xlabel('ASI', color=SEC)
ax2.set_ylabel('CSI_V3', color=SEC)
ax2.set_title(f'ASI vs CSI_V3\n(Pearson r={r:.3f}, p={p:.4f})', color=TXT, fontwe
ax2.tick_params(colors=SEC)
ax2.legend(fontsize=7, facecolor='#2a2a2d', labelcolor=TXT)

# Panel 3: Componentes del ASI por cohorte (barplot)
ax3 = fig.add_subplot(gs[1, 0])
ax3.set_facecolor(BG)

componentes = ['delta_H', 'alpha', 'F']
colores_comp = ['#A1C9F4', '#8DE5A1', '#FF9F9B']
x = np.arange(len(componentes))
width = 0.25

for i, (cohorte, color) in enumerate(COLORES_COHORT.items()):
    mask = resultado['cohorte'] == cohorte
    if mask.sum() > 0:
        medias = [resultado.loc[mask, c].mean() for c in componentes]
        ax3.bar(x + i * width, medias, width, label=cohorte, color=color, alpha=0.

ax3.set_xticks(x + width)
ax3.set_xticklabels(componentes, color=SEC)
ax3.set_ylabel('Valor medio', color=SEC)
ax3.set_title('Componentes del ASI por Cohorte', color=TXT, fontweight='bold')
ax3.tick_params(colors=SEC)
ax3.legend(fontsize=7, facecolor='#2a2a2d', labelcolor=TXT)

# Panel 4: Precision/Recall del umbral ASI > 1
ax4 = fig.add_subplot(gs[1, 1])
ax4.set_facecolor(BG)

umbrales = np.linspace(0, 5, 100)
precisiones = []
recalls_list = []
f1s = []

for umbral in umbrales:
    pred_pos = resultado['ASI'] >= umbral
    tp_ = (pred_pos & resultado['cohorte'].isin(['Elite', 'Intermediate'])).sum()
    fp_ = (pred_pos & (resultado['cohorte'] == 'Basic')).sum()
    fn_ = (~pred_pos & resultado['cohorte'].isin(['Elite', 'Intermediate'])).sum()
    prec = tp_ / (tp_ + fp_) if (tp_ + fp_) > 0 else 0
    rec = tp_ / (tp_ + fn_) if (tp_ + fn_) > 0 else 0
    f1_ = 2 * prec * rec / (prec + rec) if (prec + rec) > 0 else 0
    precisiones.append(prec)
    recalls_list.append(rec)

```

```

f1s.append(f1_)

ax4.plot(umbrales, precisiones, color='#A1C9F4', linewidth=2, label='Precision')
ax4.plot(umbrales, recalls_list, color='#8DE5A1', linewidth=2, label='Recall')
ax4.plot(umbrales, f1s, color='#FFD700', linewidth=2, label='F1')
ax4.axvline(x=1.0, color='white', linestyle='--', linewidth=1.5, alpha=0.7, label=
ax4.set_xlabel('Umbral ASI', color=SEC)
ax4.set_ylabel('Score', color=SEC)
ax4.set_title('Precision / Recall por Umbral ASI', color=TXT, fontweight='bold')
ax4.tick_params(colors=SEC)
ax4.legend(fontsize=7, facecolor='#2a2a2d', labelcolor=TXT)
ax4.set_ylim(0, 1.05)

# Panel 5: Top 20 usuarios por ASI
ax5 = fig.add_subplot(gs[1, 2])
ax5.set_facecolor(BG)

top20 = resultado.nlargest(20, 'ASI').reset_index()
colores_barras = [COLORES_COHORT.get(c, '#909094') for c in top20['cohort']]
ax5.barh(range(20), top20['ASI'].clip(upper=10), color=colores_barras, alpha=0.8)
ax5.axvline(x=1.0, color='white', linestyle='--', linewidth=1.5, alpha=0.7)
ax5.set_yticks(range(20))
ax5.set_yticklabels([f"#{i+1}" for i in range(20)], fontsize=7, color=SEC)
ax5.set_xlabel('ASI', color=SEC)
ax5.set_title('Top 20 Usuarios por ASI', color=TXT, fontweight='bold')
ax5.tick_params(colors=SEC)

from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=c, alpha=0.8, label=co) for co, c in COLORES_C]
ax5.legend(handles=legend_elements, fontsize=7, facecolor='#2a2a2d', labelcolor=T

plt.suptitle('Shadow Node Theory v2.0 – Indice de Soberania Atomica (ASI)\nFracta
            color=TXT, fontsize=12, fontweight='bold', y=1.01)

plt.savefig(OUTPUT_PNG_DIST, dpi=150, bbox_inches='tight', facecolor=BG)
plt.close()
print(f"\nFigura guardada: {OUTPUT_PNG_DIST}")

# =====
# GUARDAR CSV
# =====

cols_export = ['delta_H', 'alpha', 'F', 'ASI_raw', 'ASI', 'soberania', 'CSI_V3',
cols_disponibles = [c for c in cols_export if c in resultado.columns]
resultado[cols_disponibles].to_csv(OUTPUT_CSV, index=True)
print(f"CSV guardado: {OUTPUT_CSV}")

```

```

# =====
# TABLA RESUMEN PARA EL PAPER
# =====

print("\n" + "=" * 70)
print("TABLA RESUMEN – INTERPRETACION DEL ASI")
print("=" * 70)
print(f"\n{'Rango ASI':<20} {'Interpretacion':<35} {'N usuarios':>12} {'% del tot
print("-" * 82)

rangos = [
    (0, 0.1, "Satelizacion severa"),
    (0.1, 0.5, "Satelizacion moderada"),
    (0.5, 1.0, "Transicion pre-soberania"),
    (1.0, 2.0, "Soberania cognitiva activa"),
    (2.0, float('inf'), "Soberania avanzada / Elite"),
]

for rmin, rmax, desc in rangos:
    mask = (resultado['ASI'] >= rmin) & (resultado['ASI'] < rmax)
    n = mask.sum()
    pct = n / len(resultado) * 100
    label = f"[{rmin:.1f}, {rmax if rmax != float('inf') else '∞'}]"
    print(f"{label:<20} {desc:<35} {n:>12,} {pct:>11.1f}%")

print(f"\n{'='*70}")
print(f"Conclusion:")
print(f"  ASI = 1 como umbral empirico calibrado sobre mediana Elite")
print(f"  F1 del clasificador ASI>1 vs Elite/Intermediate: {f1:.4f}")
print(f"  Correlacion Spearman ASI-CSI_V3: rs={rs:.4f}, p={ps:.6f}")
print(f"  El ASI captura una dimension de soberania NO contenida en el CSI_V3:")
print(f"  usuarios con ASI alto pero CSI_V3 bajo son candidatos a leapfrog latent
print(f"{'='*70}")

print("\nArchivos generados:")
print(f"  - {OUTPUT_CSV}")
print(f"  - {OUTPUT_PNG_DIST}")

```